



ज्ञानेन विकासः

S. R. Mahadeva Prasanna

Professor, Dept of DSAI & Director, IIIT Dharwad

Indian Institute of Information Technology Dharwad

Dharwad 580009, Karnataka

Some Good Resources for DL

- Duda and Hart, "Pattern Classification" Wiley, 2001.
- C. M. Bishop, "Neural Networks for Pattern Recognition", 1995.
- B. Yegnanarayana, "Artificial Neural Networks", PHI, 1999.
- C. M. Bishop, "PRML", Springer, 2006.
- Michael Nielsen "Neural Networks and Deep Learning" Open Book.
- Ian Goodfellow, et. al. , "Deep Learning", MIT Press, 2016.
- Charu Agarwal, "NNDL", Springer 2018.
- Mithesh M. Khapra, "Deep Learning Part I and II" NPTEL Course.
- Ali Ghodsi, "Deep Learning", University of Waterloo Course.
- C. M. Bishop & H. Bishop, "Deep Learning", Springer, 2024



How Deep Learning Came ?

- At the turn of the century, **several advances** again brought popularity to neural networks.
- Not all of these advances were algorithm centric.
- Increased **data availability and computational power** have played the primary role in this resurrection.
- However, some **changes to basic backpropagation algorithm** and **clever methods for initialization**, such as pretraining, have also helped.
- Easier to perform intensive experimentation required for making algorithmic adjustments due to reduced testing cycle times.
- Therefore, **increased data, computational power, and reduced experimentation time** (for algorithmic tweaking) went hand-in-hand.



About Deep Learning

- Deep learning provides a powerful framework for supervised learning.
- By adding more layers and more units within a layer, a deep network can represent **functions of increasing complexity**.
- Tasks easy for a person to do effortlessly, can now be accomplished via deep learning, **given sufficiently large datasets and hence large models**.
- Tasks difficult for a person and need time to think and reflect in order to accomplish the task **like creative content generation are under exploration**.
- **Core function approximation** is behind nearly all modern practical applications of deep learning.



About Deep FFNN

- Goal of a feedforward network is to approximate some function $f^*(\cdot)$.
Ex. in case of 2 i/p binary, 16 fns possible (OR, AND, XOR, ...).
- A classifier, $y = f^*(\mathbf{x})$ maps an input $\mathbf{x}$ to a class y .
- FFNN defines a mapping $\mathbf{y} = f(\mathbf{x}; \mathbf{w})$ and learns the value of parameters $\mathbf{w}$ that result in the best function approximation.
- FFNN may be viewed to be composing of different functions:
 $f(\mathbf{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x})))$, $f^{(1)}$ first, $f^{(2)}$ second layer, ...
- The overall length gives the depth of model.
- It is from this terminology that the name **deep learning** arises.



Interpreting FFNN as a Function Approximator

- Purpose of neural network training is to drive $f(\mathbf{x})$ to match $f^*(\mathbf{x})$.
- Training data provides noisy, approximate examples of $f^*(\mathbf{x})$ evaluated at different training points.
- Each example $\mathbf{x}$ is accompanied by a label $y = f^*(\mathbf{x})$.
- Training examples specify directly what the output layer must do at each point $\mathbf{x}$; it must produce a value that is close to y .
- The behavior of other layers is not directly specified by training data.
- **The learning algorithm must decide how to use those layers to produce the desired output.**



Vanishing or Exploding Gradient Problem

- Unstable behavior that may be encountered when training a deep neural network (DNN).
- In a DNN, backpropagated error signal typically decreases (or increases) exponentially as a function of distance from output layer.
- Therefore DNN is unable to back propagate useful gradient information termed as **exponentially small (vanishing gradient) or exponentially large (exploding) gradient** problem.
- Results in inability of models with many layers to learn on a given pattern information or **prematurely converge to a poor solution (local minima)**.



Addressing Vanishing or Exploding Gradient Problem

- Alternate **weight initialization** and alternate **activation function**.
- Alternate weight initialization scheme: **unsupervised pre-training**, **layer-wise training**, and variations on gradient descent.
- Alternate activation function: Use of **rectified linear unit (ReLU) activation function**, instead of hyperbolic tangent.
- ReLU acts **like a linear function**, easier to train and less likely to saturate, **but a nonlinear function**, forcing negative inputs to 0.

Ref: Vanishing/Exploding Gradients (C2W1L10) in deeplearning.ai by Andrew Ng

Weight Initialization in a Deep Network (C2W1L11)



Rectified Linear Units and Their Generalizations

- **Sigmoidal units saturate** quickly, and are only strongly **sensitive to** when input z is near 0.
- **widespread saturation** of sigmoidal units can make gradient based learning very difficult.
- Rectified linear units (**ReLU**) use activation function $g(z) = \max\{0, z\}$.
- ReLU is easy to optimize, so similar to linear units.
- Only diff. between a linear unit and a ReLU is, a ReLU outputs zero across half its domain.
- **Makes derivatives** through a ReLU **large** whenever unit is active.
- The gradients are not only large but also consistent.

PS: Saturation and differential natures of sigmoidal & Relu

Rectified Linear Units and Their Generalizations

- ReLU is used on top of affine transformation: $\mathbf{h} = \mathbf{g}(\mathbf{W}^T \mathbf{x} + \mathbf{b})$
- During initialization, set all elements of $\mathbf{b}$ to a small, positive value, say 0.1.
- ReLU will be initially active for most inputs in training set and allow derivatives to pass through.
- One drawback to ReLU is that they cannot learn via gradient based methods on examples for which their activation is zero.
- A variety of generalizations of ReLU guarantee that they receive gradient everywhere.



Affine transformation preserves lines and parallelism. A function which maps an affine space onto itself while preserving both the dimension of any affine subspaces. Points to points, lines to lines, planes to planes, and so on.

Rectified Linear Units and Their Generalizations

- Three generalizations of ReLU are based on using a non-zero slope α_i when $z_i < 0$: $h_i = g(z, \alpha)_i = \max(0, z_i) + \alpha_i \min(0, z_i)$.
- **Absolute value rectification** fixes $\alpha_i = 1$ to obtain $g(z) = |z|$.
- A **leaky ReLU** fixes α_i to a small value like 0.01.
- A **parametric ReLU** or PReLU treats α_i as a learnable parameter.
- **Maxout units** generalize rectified linear units further.
- Instead of applying an element-wise function $g(z)$, maxout units divide z into groups of k values.
- Each maxout unit then outputs the maximum element of one of these groups: $g(z)_i = \max_{j \in G(i)} z_j$



Rectified Linear Units and Their Generalizations

- Where $G^{(i)}$ is the set of indices into the inputs for group i .
- This provides a way of learning a piecewise linear function that responds to multiple directions in the input $\mathbf{x}$ space.
- A maxout unit can learn a piecewise linear, convex function with up to k pieces.
- With large enough k , a maxout unit can learn to approximate any convex function with arbitrary fidelity.



About Breakthrough in Deep Learning

- DNNs have many levels of non-linearities, can potentially represent highly non-linear and highly-varying functions.
- DNNs were generally found to be not better, and often worse, than shallow neural networks with one or two hidden layers.
- A reasonable explanation is that **gradient-based optimization starting from random initialization often get stuck in poor solutions** for such deep networks.
- Hinton et al. in 2006 introduced a greedy layer-wise unsupervised learning algorithm as a principle to solve the problem of training deep networks gave much needed breakthrough.



Tesauro, G. (1992). Practical issues in temporal difference learning. Machine Learning, 8, 257–277.

Hinton, G. E., Osindero, S., Teh, Y. (2006). A fast learning algorithm for deep belief nets. Neural Computation. ▶



Unsupervised Greedy Layer-wise Training

- Upper layers of a Deep network are supposed to represent more “abstract” concepts that explain the input observation x .
- The earlier lower layers extract “low-level features” from x .
- They learn simpler concepts first, and build on them to learn more abstract concepts.
- Important aspects of greedy layer-wise training:
 - Pre-training one layer at a time in a greedy way.
 - Greedy $\implies$ each layer is pre-trained to model its input, ignoring the effect of higher layers.
 - Using learning at each layer in order to preserve information from the input.
 - Fine-tuning the whole network with respect to the ultimate criterion of interest.

Findings from Unsupervised Greedy Layerwise Training

- MNIST handwritten digit recognition task
- 0.1% changes in performance is statistically significant
- Proposed generative model (784-500-500-2000-10) $\implies$ 1.25%
- Support Vector Machine $\implies$ 1.4%
- Back-prop (784-500-300-10, cross-entropy & weight-decay) $\implies$ 1.51
- Backprop (784-800-10, corss-entropy & early stopping) $\implies$ 1.53
- Backprop (784-500-15-10, squared-error & online updates) $\implies$ 2.95

Greedy Layer-Wise Supervised Training of Deep Networks

- Train each new hidden layer as the hidden layer of a one-hidden layer supervised neural network.
- Then throw away the output layer and use the trained hidden layer as pre-training initialization for next hidden layer.
- Repeat this process taking as input the output of the previously trained layers.
- Final fine-tuning by adding a logistic regression layer on top of network and training whole network by stochastic gradient descent on cross-entropy with respect to target classification.



Yoshua Bengio et. al., 2006, Greedy Layer-Wise Training of Deep Networks, Proc. NIPS 2006.

Results from Supervised Greedy Layerwise Training

Figure: MNIST Digits Recognition task

	Experiment 2			Experiment 3		
	train.	valid.	test	train.	valid.	test
DBN, unsupervised pre-training	0%	1.2%	1.2%	0%	1.5%	1.5%
Deep net, auto-associator pre-training	0%	1.4%	1.4%	0%	1.4%	1.6%
Deep net, supervised pre-training	0%	1.7%	2.0%	0%	1.8%	1.9%
Deep net, no pre-training	.004%	2.1%	2.4%	.59%	2.1%	2.2%
Shallow net, no pre-training	.004%	1.8%	1.9%	3.6%	4.7%	5.0%

- Any change in 0.1% in perf is statistically significant
- Expt-2 shows for training data even shallow network gives good performance on train data (overfitting)
- Expt-3 shows, if we reduce no. of nodes in hidden layers towards to O/L, then perf degrades for shallow or w/o pre-training deep cases



Deep Neural Networks for Acoustic Modeling

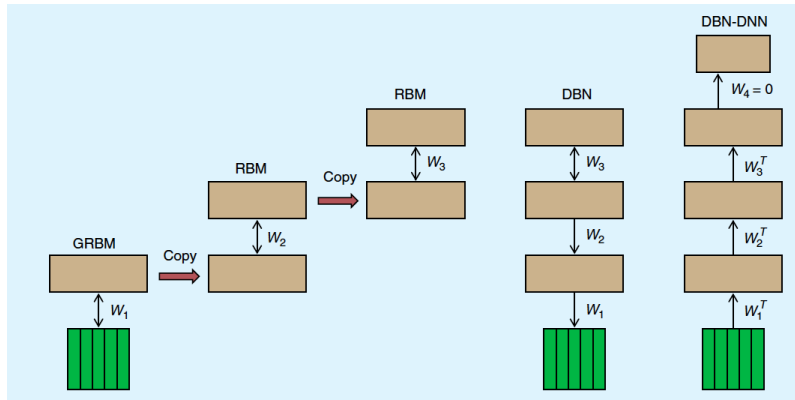


Figure: Acoustic Modeling using DNN



Hinton et. al, "Deep Neural Networks for Acoustic Modeling in Speech Recognition", IEEE Signal Processing Magazine, Nov.

DNN Acoustic Modeling using TIMIT Database

Figure: Acoustic Modeling using DNN using TIMIT

[TABLE 1] COMPARISONS AMONG THE REPORTED SPEAKER-INDEPENDENT (SI) PHONETIC RECOGNITION ACCURACY RESULTS ON TIMIT CORE TEST SET WITH 192 SENTENCES.

METHOD	PER
CD-HMM [26]	27.3%
AUGMENTED CONDITIONAL RANDOM FIELDS [26]	26.6%
RANDOMLY INITIALIZED RECURRENT NEURAL NETS [27]	26.1%
BAYESIAN TRIPHONE GMM-HMM [28]	25.6%
MONOPHONE HTMS [29]	24.8%
HETEROGENEOUS CLASSIFIERS [30]	24.4%
MONOPHONE RANDOMLY INITIALIZED DNNs (SIX LAYERS) [13]	23.4%
MONOPHONE DBN-DNNs (SIX LAYERS) [13]	22.4%
MONOPHONE DBN-DNNs WITH MMI TRAINING [31]	22.1%
TRIPHONE GMM-HMMs DT W/ BMMI [32]	21.7%
MONOPHONE DBN-DNNs ON FBANK (EIGHT LAYERS) [13]	20.7%
MONOPHONE MCRBM-DBN-DNNs ON FBANK (FIVE LAYERS) [33]	20.5%

DNN ASR using SWB and RT Databases

Figure: ASR using DNN using Hub and RT

[TABLE 2] COMPARING FIVE DIFFERENT DBN-DNN ACOUSTIC MODELS WITH TWO STRONG GMM-HMM BASELINE SYSTEMS THAT ARE DT. SI TRAINING ON 309 H OF DATA AND SINGLE-PASS DECODING WERE USED FOR ALL MODELS EXCEPT FOR THE GMM-HMM SYSTEM SHOWN ON THE LAST ROW WHICH USED SA TRAINING WITH 2,000 H OF DATA AND MULTIPASS DECODING INCLUDING HYPOTHESES COMBINATION. IN THE TABLE, "40 MIX" MEANS A MIXTURE OF 40 GAUSSIANS PER HMM STATE AND "15.2 NZ" MEANS 15.2 MILLION, NONZERO WEIGHTS. WERs IN % ARE SHOWN FOR TWO SEPARATE TEST SETS, HUB500-SWB AND RT03S-FSH.

MODELING TECHNIQUE	#PARAMS [10 ⁶]	WER	
		HUBS'00-SWB	RT03S-FSH
GMM, 40 MIX DT 309H SI	29.4	23.6	27.4
NN 1 HIDDEN-LAYER $\times$ 4,634 UNITS	43.6	26.0	29.4
+ 2 $\times$ 5 NEIGHBORING FRAMES	45.1	22.4	25.7
DBN-DNN 7 HIDDEN LAYERS $\times$ 2,048 UNITS	45.1	17.1	19.6
+ UPDATED STATE ALIGNMENT	45.1	16.4	18.6
+ SPARSIFICATION	15.2 NZ	16.1	18.5
GMM 72 MIX DT 2000H SA	102.4	17.1	18.6

Hinton et. al, "Deep Neural Networks for Acoustic Modeling in Speech Recognition",

IEEE Signal Processing Magazine, Nov. 2012

DNN ASR using Large Vocabulary Databases

Figure: ASR using DNN using large vocabulary database

[TABLE 3] A COMPARISON OF THE PERCENTAGE WERs USING DNN-HMMs AND GMM-HMMs ON FIVE DIFFERENT LARGE VOCABULARY TASKS.

TASK	HOURS OF TRAINING DATA	DNN-HMM	GMM-HMM WITH SAME DATA	GMM-HMM WITH MORE DATA
SWITCHBOARD (TEST SET 1)	309	18.5	27.4	18.6 (2,000 H)
SWITCHBOARD (TEST SET 2)	309	16.1	23.6	17.1 (2,000 H)
ENGLISH BROADCAST NEWS	50	17.5	18.8	
BING VOICE SEARCH (SENTENCE ERROR RATES)	24	30.4	36.2	
GOOGLE VOICE INPUT	5,870	12.3		16.0 (> 5,870 H)
YOUTUBE	1,400	47.6	52.3	

Hinton et. al, "Deep Neural Networks for Acoustic Modeling in Speech Recognition",

IEEE Signal Processing Magazine, Nov. 2012



Summary of Transition from Shallow to Deep

- Neural network training as function approximation.
- DNNs have many levels of non-linearities and can represent very compactly highly non-linear and highly-varying functions.
- Upper layers represent more “abstract” concepts, and earlier lower layers extract “low-level features” about input data.
- Learn simpler concepts first, and build on them to learn more abstract concepts.
- Alternative activation function.
- Alternative weight initialization scheme.
- Greedy layer-wise training.



- Hinton et. al., "A fast learning algorithm for deep belief nets" Neural Computation 2006.
- Yoshua Bengio et. al., "Greedy Layer-Wise Training of Deep Networks", NIPS 2006
- Hinton et. al., "Deep Neural Networks for Acoustic Modeling in Speech Recognition", IEEE Signal Processing Magazine, Nov. 2012

